



FDI Analytics

Quick-Start Guide

A picture-led, one-pass walkthrough for operators

This short guide takes you through one complete run of the toolbox, one screen at a time, using real screenshots. For the full reference — every setting, the deep-learning options, the glossary — see the complete *FDI Analytics User Guide*.

NOTE

The whole job in six steps: 1) load your data → 2) clean it → 3) extract features → 4) detect faults → 5) identify the cause → 6) download the report. Work top-to-bottom down the left sidebar; the status chip in the top-right tells you how far you've got.

Step 1 • Home: Load Your Data

1 Upload a CSV or Excel file, then check it loaded correctly

The screenshot shows the 'Home • Data Explorer' page. At the top right, there's a green 'Detection complete' chip and a purple 'AI Assistant' button. The main section is titled 'Data Explorer' with a folder icon. Below it, instructions say 'Upload your dataset to get started. Inspect the structure, review statistics, and visualize any variable before moving to processing.' There's an 'Upload Dataset' section with a folder icon and text 'CSV or Excel files supported'. A large dashed box in the center contains a file upload icon and the text 'Drag & drop your file here or click to browse'. Below this are three buttons: '.csv', '.xls', and '.xlsx'. At the bottom, a green banner confirms the upload: '✓ Loaded "AUS_df.csv" — 1,500 rows × 10 columns'.

The Home (Data Explorer) page. Drag a file onto the upload box — or click [click to browse](#). A green banner confirms the load (here: *AUS_df.csv*, 1,500 rows × 10 columns), and the top-right chip updates.

- 1 Drag your file onto the dashed box, or click [click to browse](#). Supported: .csv, .xls, .xlsx.
- 2 Wait for the green **Loaded...** confirmation banner.
- 3 Scroll down to confirm the data looks right (next two screenshots).

**Data Preview**

First 10 rows of the uploaded dataset

AC1_PV	AC1_OP	FC1_PV	FC1_OP	PC1_PV	PC1_OP	PC2_PV	PC2_OP
-0.5843340339841737	-1.2524783653293894	-0.4737545802121951	0.0310575807826952	0.9521486055381432	-0.6593497860981423	-2.0149728196646968	0.0149728196646968
-0.4871147414829822	-1.311911552951079	-0.4308347997168982	-0.2177063877834726	0.8612024071783432	-0.3994477831890707	-2.4050172350632675	0.0149728196646968
-0.2726266353632751	-1.4547769910882664	-0.6443074871531858	-0.4281693022615843	0.5718887416107047	-0.223853357405786	-2.4050172350632675	0.0149728196646968
-0.0124005591946723	-1.631801945150174	-0.4169081893417846	-0.8051729152348138	0.4560870900865828	-0.0675277899817846	-2.9899434534040275	0.0149728196646968
0.293981185379437	-1.844261023939963	-0.3391718829087025	-1.1419466440579973	0.1916288776832661	0.0062097309836619	-2.7950616504619017	0.0149728196646968
0.6376425082810709	-2.087019374850615	-0.4076237824250421	-1.670060315030437	-0.2878623356587608	-0.0359187703635379	-3.3799878688026617	0.0149728196646968
1.0107515653237746	-2.353849623443825	-0.4296673816874264	-2.2077630268824757	-0.618506524878916	-0.1606353181692926	-3.1851060658604724	-0.0149728196646968
1.4070428811583844	-2.6408915259157357	-1.6177143744567608	-2.38383765682685	-0.924390492556088	-0.3555491767432513	-3.3799878688026617	-0.0149728196646968
1.7217785928647122	-2.876075058796172	-2.1525566439120727	-2.1369474088923277	-1.1640884834164429	-0.5803019498368301	-2.9899434534040275	-0.0149728196646968
2.00591789994929	-3.0935597380098545	-2.1479144404537016	-1.9073615220853597	-1.494827904257915	-0.8929025102534456	-2.5998990380054567	-0.0149728196646968

***Data Preview** — the first 10 rows, exactly as the toolbox read them. Use it to confirm the right file loaded and the columns (here *AC1_PV*, *AC1_OP*, *FC1_PV*, ...) are the measurements you expect.*

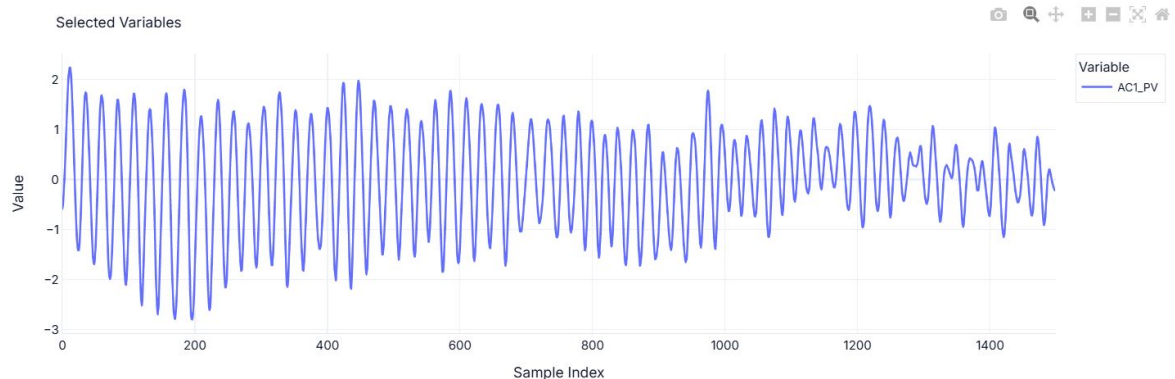
**Variable Explorer**

Plot one or more numeric columns

SELECT VARIABLES:

AC1_PV

X v



***Variable Explorer** — pick any numeric column from the drop-down to plot it over time. A quick eyeball here often reveals an obviously broken sensor before you go any further.*

CAUTION

Only **numeric** columns are used later. Make sure rows are already in time order — the toolbox treats row order as time order.

Step 2 • Pre-processing: Clean the Signal

2 Optionally tidy the data, then compare before vs. after

 **Processing Pipeline**
Enable steps, choose methods, then click Apply

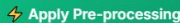
 Step 1: Missing Value Handling

 Step 2: Outlier Removal

 Step 3: Detrending

 Step 4: Noise Filtering / Smoothing

 Step 5: Normalization / Scaling

 **Apply Pre-processing**

 **⚠ No steps enabled — enable at least one step above.**

*The **Processing Pipeline**. Five optional clean-up steps, applied top-to-bottom. Click a step to expand it, tick **Enable this step**, choose a method — then click the green **Apply Pre-processing**. (Here nothing is enabled yet, so the amber banner reminds you to switch on at least one step.)*

- 1 Click a step's bar to expand it, then tick **Enable this step**.
- 2 Choose a method and set its number if it has one.
- 3 Repeat for any other steps, then click **Apply Pre-processing**.

TIP

Not sure what to turn on? For typical noisy data, **Outlier Removal** + **Noise Filtering** is a safe starting pair. Every step is optional — if your data is already clean, skip straight to Step 3.

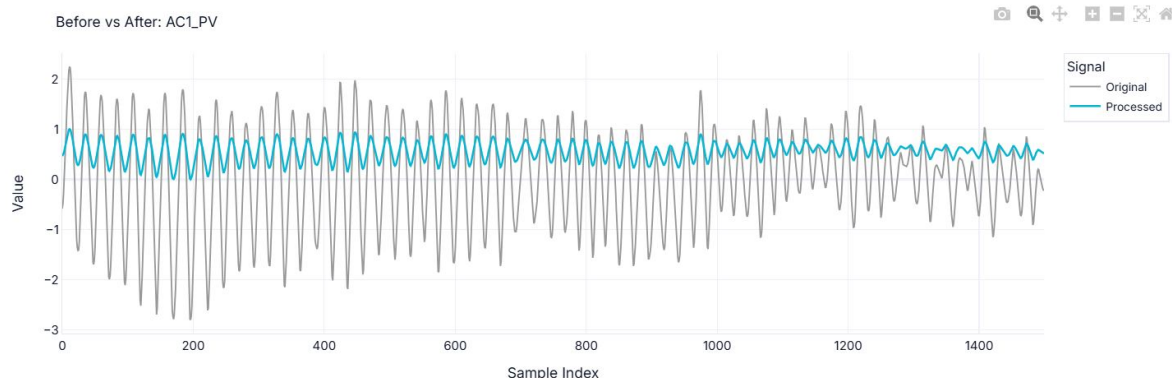


Before vs After Comparison

Compare original and processed signal for a selected variable

SELECT VARIABLE:

AC1_PV



Before vs. After Comparison — pick a variable to see the original signal (grey) overlaid with the cleaned-up version (teal). Here smoothing has clearly tamed the noise while keeping the underlying shape. Use this to confirm the clean-up did what you intended.

Step 3 • Feature Extraction

3

Summarise the signal into per-window features



Feature Extraction

Slide a window across your data and compute features per window. Choose traditional statistical/frequency-domain features, or train a small deep learning autoencoder per variable and use its learned embedding instead.



Extraction Settings

Configure data source, variables, method family, and window parameters

DATA SOURCE:



Pre-processed data (recommended)



Raw (original) data

VARIABLES TO EXTRACT FROM:

AC1_PV



METHOD FAMILY:



Traditional — statistical & frequency-domain



Deep Learning — learned embeddings

FEATURES TO COMPUTE:



Mean



Standard Deviation



RMS

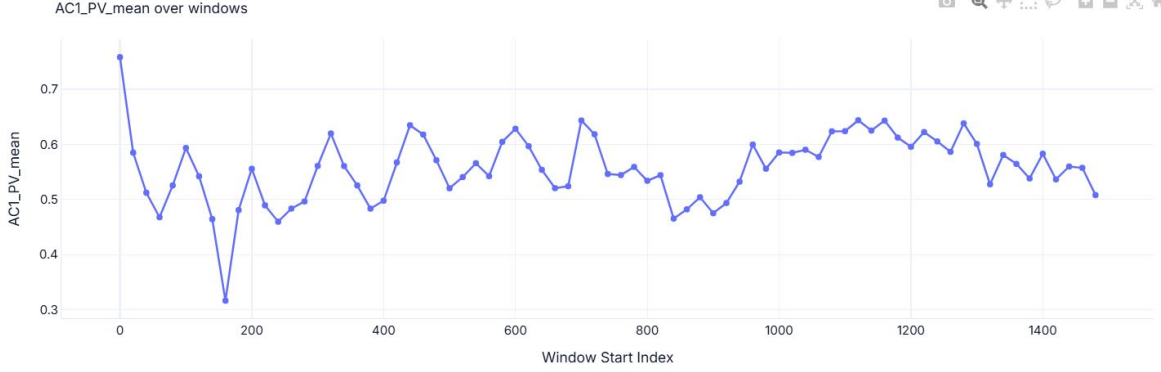
Extraction Settings. Choose the data source (**Pre-processed** recommended), the variables, and the **Method Family** — **Traditional** (tick the statistics you want, e.g. Mean, Std, RMS) or **Deep Learning** (a learned embedding). Set the window parameters lower down, then click **Extract Features**.

- 1 Pick the data source and the variables to extract from.
- 2 Choose **Traditional** and tick features (or **Deep Learning**).

3 Set window size / step size, then click **Extract Features**.


Feature Time-Series
 Visualize how a feature evolves across windows

AC1_PV_mean



← Back to Pre-processing

Next: Fault Detection →

Extracted Features Table — one row per window, with the computed columns (e.g. *AC1_PV_mean*, *AC1_PV_std*, *AC1_PV_rms*). Page through with the arrows, or **Download Features (CSV)** to keep a copy.


Extracted Features Table
 Computed feature columns per window

WINDOW_START	WINDOW_END	AC1_PV_MEAN	AC1_PV_STD	AC1_PV_RMS
0	20	0.7581	0.1826	0.7798
20	40	0.5849	0.2307	0.6287
40	60	0.5122	0.2151	0.5555
60	80	0.4678	0.2371	0.5244
80	100	0.5254	0.2665	0.5891
100	120	0.5934	0.2459	0.6423
120	140	0.5421	0.2398	0.5927
140	160	0.4644	0.3106	0.5587
160	180	0.3165	0.2449	0.4002
180	200	0.4808	0.3549	0.5976

Download Features (CSV)

Feature Time-Series — pick any single feature to watch how it evolves window by window. A sudden change here is an early visual hint of where a fault may sit. When ready, click **Next: Fault Detection** (bottom-right) to continue.

Step 4 • Fault Detection

4 Flag the unusual windows — no labelled faults needed

Detection Configuration
Select a method family and tune hyperparameters

METHOD FAMILY:

☒ Traditional / Classical ML ☐ Deep Learning

DETECTION SENSITIVITY (USED BY EVERY METHOD):
Contamination — expected fraction of anomalous windows (0.01 – 0.5):

– 0.05 +

DETECTION ALGORITHM:

☒ PCA-based (Hotelling's T^2) ☐ Isolation Forest ☐ One-Class SVM ☐ Local Outlier Factor (LOF)

PCA — n components: – 2 + iForest — n estimators: – 100 +

OC-SVM — kernel: rbf × LOF — n neighbors: – 20 +

☒ Standardize features before detection (recommended)

Run Fault Detection

*Detection Configuration (Traditional). Pick a **Method Family**, set the **Contamination** sensitivity, and choose an **Algorithm** — PCA, Isolation Forest, One-Class SVM, or LOF. Each has its own dials (here *n* components, *n* estimators, kernel, *n* neighbors); the defaults are sensible. Keep **Standardize features** ticked, then click **Run Fault Detection**.*

- 1 Choose the **Method Family** and an **Algorithm** (the default PCA is a fine place to start).
- 2 Set **Contamination** — your estimate of the abnormal fraction (start at 0.05); leave **Standardize features** on.
- 3 Click **Run Fault Detection**.

TIP

Contamination is the most direct dial you have: if *nothing* is flagged, raise it; if *almost everything* is flagged, lower it.



Detection Configuration

Select a method family and tune hyperparameters

METHOD FAMILY:



Traditional / Classical ML



Deep Learning

DETECTION SENSITIVITY (USED BY EVERY METHOD):

Contamination — expected fraction of anomalous windows (0.01 – 0.5):



0.05



DEEP LEARNING ALGORITHM:



Autoencoder — reconstruction-error anomaly score



Variational Autoencoder — probabilistic reconstruction error



Deep SVDD — one-class hypersphere distance

Latent dimension:



4



Training epochs:



150



VAE — KL weight (β):



0.5



Standardize features before detection (recommended)



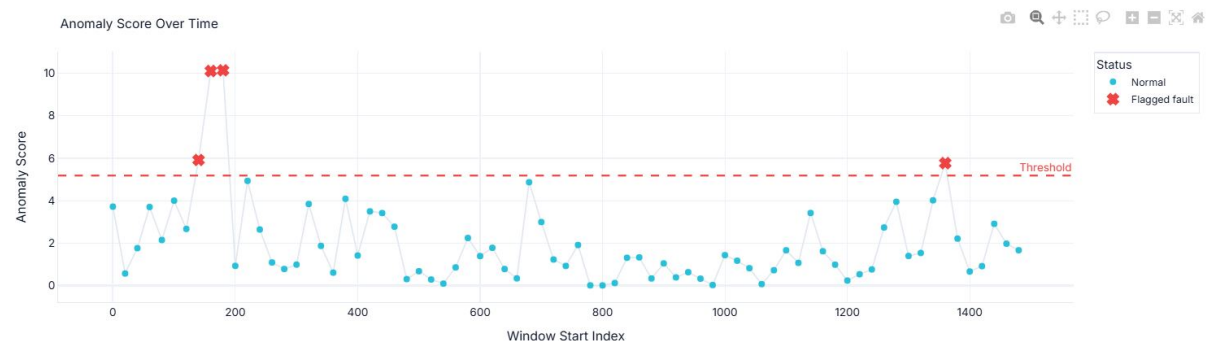
Run Fault Detection

Same card with **Deep Learning** selected instead — the algorithm list switches to Autoencoder, Variational Autoencoder, and Deep SVDD, with extra training dials (**Latent dimension**, **Training epochs**, and the VAE **KL weight**). Everything downstream works the same; this just trades a little training time for a learned model.



Detection Results

Anomaly scores and flagged windows



FLAGGED WINDOWS:

WINDOW_START	WINDOW_END	ANOMALY_SCORE
140	160	5.9164
160	180	10.1087
180	200	10.1406
1360	1380	5.7663

Detection Results. Every window's **Anomaly Score** is plotted over time: blue dots are normal, red crosses are **Flagged fault**, and the dashed line is the **Threshold**. The **Flagged Windows** table below lists each flagged window's start, end, and score — your shortlist to investigate next.

Step 5 • Fault Identification

5 See which variable and feature is most responsible

Fault Identification

Contribution analysis reveals which variables and features deviated most from normal behavior in flagged windows — a pointer toward the root cause, regardless of which detection algorithm was used. Choose a traditional z-score baseline, or a deep learning autoencoder-based method below.

✓ Analyzed 4 flagged window(s) using  Autoencoder Residual — per-feature reconstruction error vs. normal.



Contribution Method

Choose how feature/variable deviations are scored



Traditional — z-score vs. normal baseline



Deep Learning — autoencoder-based

DEEP LEARNING METHOD:



Autoencoder Residual — per-feature reconstruction error vs. normal



Gradient × Input Saliency — sensitivity-weighted deviation

Latent dimension:



4



Training epochs:



120



▶ Train & Run Deep Identification

✓ Deep identification complete — analyzed 4 flagged window(s) using  Autoencoder Residual — per-feature reconstruction error vs. normal.

Contribution Method. Pick how deviations are scored: **Traditional** (z-score vs. a normal baseline — instant, no button) or **Deep Learning** (autoencoder-based). For the deep-learning option, choose a method and click **Train & Run Deep Identification**; the green banner confirms when it has analysed the flagged windows.

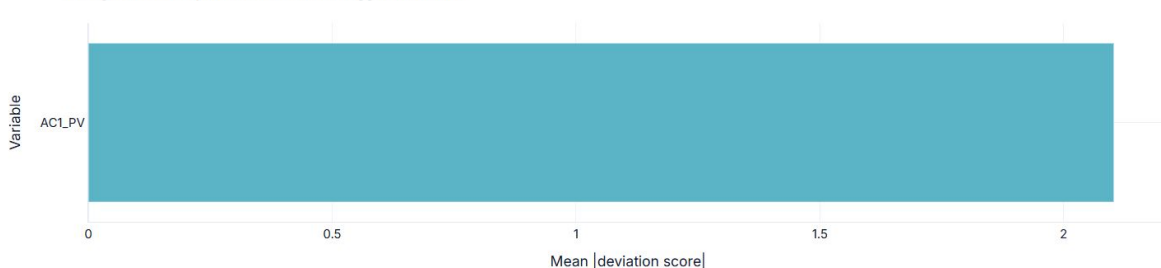
- 1 Make sure **Detection** has run — this page works from its flagged windows.
- 2 Pick a **Contribution Method**. Traditional results appear automatically; for Deep Learning, click **Train & Run Deep Identification**.
- 3 Read the charts below to see which variable and feature deviated most.
- 4 Use the **Window Drill-down** to inspect a single flagged window.



Variable Contributions

Which variables are most implicated across all flagged windows?

Average deviation by variable (across all flagged windows)

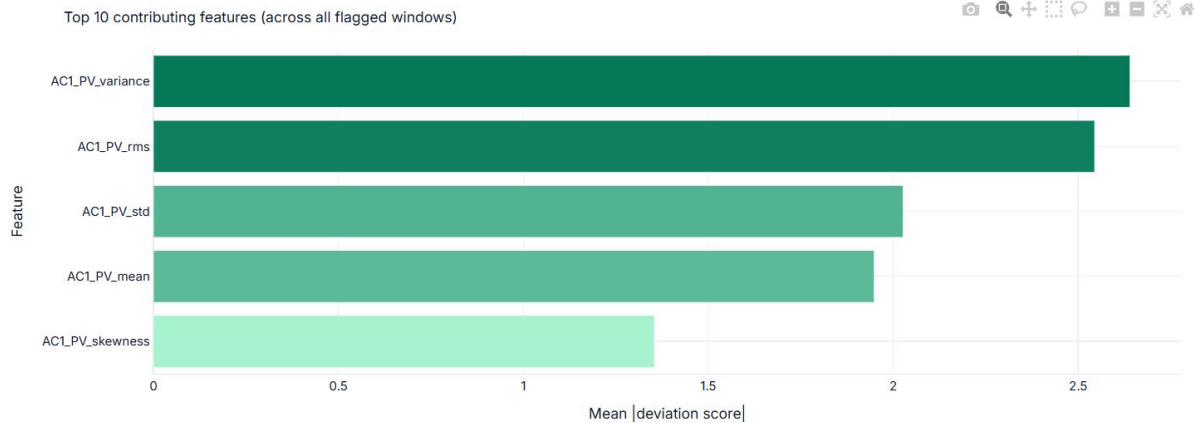


Variable Contributions — ranks your original sensor variables by how implicated they are across all flagged windows. The longest bar is the prime suspect (here **AC1_PV**).



Top Contributing Features

Top 10 features with highest average deviation from normal



Top Contributing Features — the same idea at a finer level: the ten specific features that deviated most from normal (here *AC1_PV_variance* and *AC1_PV_rms* lead).

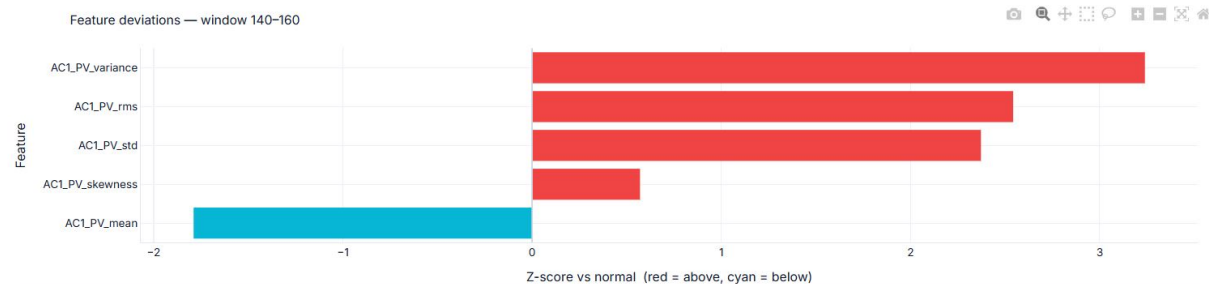


Window Drill-down

Inspect feature deviations for a specific flagged window

SELECT A FLAGGED WINDOW:

Window 140-160 (score=5.92)



FEATURE	VALUE	NORMAL MEAN	SCORE
AC1_PV_variance	2.6006	0.8374	3.2392
AC1_PV_rms	1.6827	0.9095	2.5428
AC1_PV_std	1.6126	0.859	2.3725
AC1_PV_mean	-0.4807	0.0294	-1.789
AC1_PV_skewness	0.0752	-0.0886	0.5705

Window Drill-down — pick any flagged window to see exactly what was off. Red bars sit above the normal baseline, cyan below; the table gives each feature's value, the normal mean, and its z-score.

CAUTION

A flagged window is a *starting point for investigation*, not a confirmed fault. Always check Identification before raising a maintenance action.

Step 6 • Evaluation & Report

6 Review the summary and export it

Evaluation & Report

Quantitative summary of detection performance: score distributions, contribution heatmap, and top implicated variables. Download a full CSV or HTML report when ready.

✓ Report ready — 4 of 75 windows flagged (5.3%) using PCA-based (Hotelling's T^2).



Detection Summary

Key metrics from the last detection run

75

TOTAL WINDOWS

4

FLAGGED WINDOWS

5.3%

FAULT RATE

5.183

SCORE THRESHOLD

1.973

MEAN SCORE

10.141

MAX SCORE

1.978

SCORE STD DEV

Algorithm: PCA-based (Hotelling's T^2)
Contamination parameter: 0.05

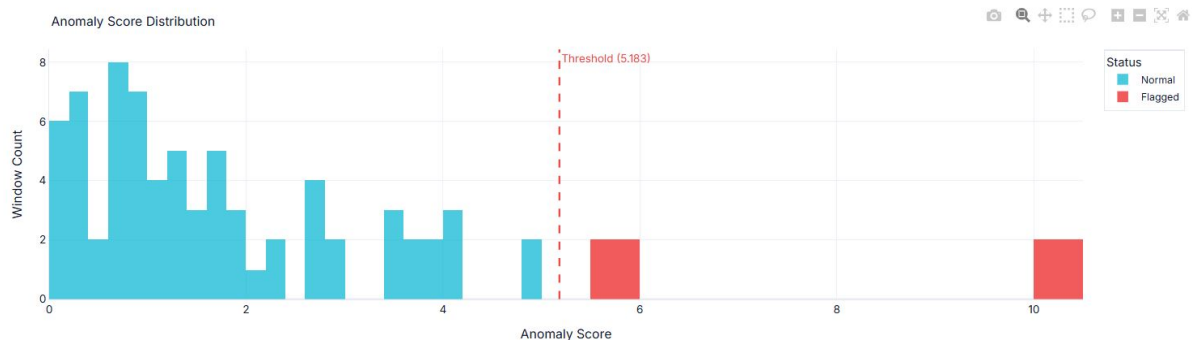
Detection Summary. Headline numbers from the last run at a glance — total windows, how many were flagged, the fault rate, the score threshold, and score statistics — plus the algorithm and contamination used. The green banner confirms the report is ready (here 4 of 75 windows flagged, 5.3%, using PCA).

- 1 Read the **Detection Summary** numbers to size up the result.
- 2 Scan the charts below — distribution, heatmap, and top variables.
- 3 Click **Download CSV** for the data, or **Download HTML Report** for a printable summary to attach to a ticket or email.



Anomaly Score Distribution

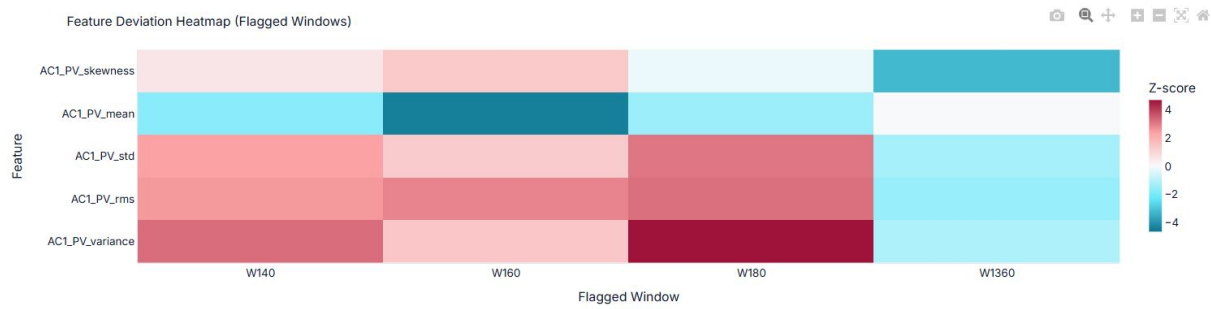
Histogram of scores — normal vs. flagged windows overlaid



Anomaly Score Distribution — a histogram of every window's score, normal (cyan) vs. flagged (red), with the dashed **Threshold** line. A clean gap between the two colours means the detector separated normal from abnormal confidently.

**Feature Deviation Heatmap**

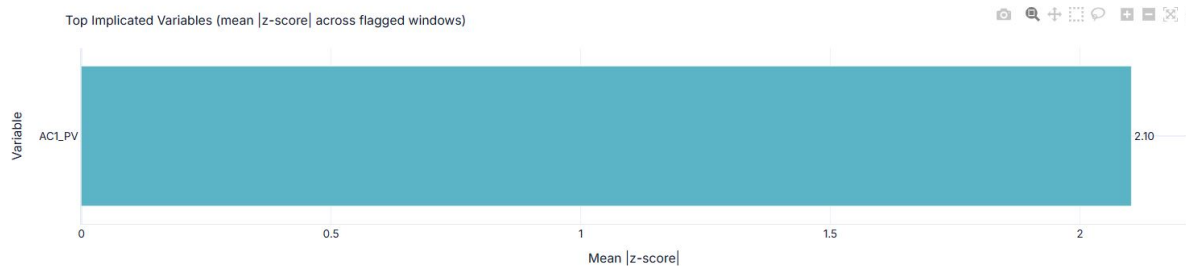
Z-score of each feature relative to normal, for every flagged window (traditional method)



Feature Deviation Heatmap — one column per flagged window, one row per feature, coloured by z-score (warm = above normal, cool = below). It shows, at a glance, which features drove each flagged window.

**Top Implicated Variables**

Ranked by mean absolute z-score across all flagged windows (traditional method)

**Download Report**

Export all results as a CSV data file or a printable HTML report

[Download CSV](#)
[Download HTML Report](#)

Top Implicated Variables and Download Report. The ranking repeats the prime-suspect variable for the record, and the two buttons export everything — **Download CSV** (raw data) or **Download HTML Report** (formatted, printable).

The AI Assistant • optional helper

The violet **AI Assistant** button (top-right, every page) opens a chat helper that already knows your data and results. It interprets results and suggests next steps — it never changes your data or settings.

 **AI Assistant**
Chat with an AI that has full context of your analysis — ask it to interpret results, recommend methods, explain faults, or guide you through any step of the workflow.

⚙️ API Configuration

PROVIDER:

Ollama (local models)

MODEL:

llama3

BASE URL:

http://localhost:11434/v1

Save Configuration

✓ Saved

*API Configuration (one-time). Pick a **Provider** (here Ollama, running locally), set the model and base URL, and click **Save Configuration**. Your IT/engineering contact can supply these details if your organisation already has an AI service.*

- 1 Open **API Configuration**, choose a provider, and **Save Configuration**.
- 2 Click a **Quick Prompt**, or type your own question.
- 3 Read the reply — it draws on the live **Analysis Context** below.

 **Quick Prompts**
Click any button to send a pre-built question

🔍 Interpret detection results

⚙️ Recommend preprocessing

📄 Explain fault features

🔍 Compare detection methods

💬 Traditional vs. deep learning

📄 Summarize findings

🔍 Suggest next steps

📄 Optimize window parameters

Quick Prompts — one-click, pre-built questions such as Interpret detection results, Recommend preprocessing, or Traditional vs. deep learning. The fastest way to get a useful answer without typing.

Conversation
Ask anything about your data or analysis

Clear

Based on my dataset, what preprocessing steps do you recommend and why?

Congratulations on your effort to preprocess your data! Given the characteristics of your dataset, I'll provide some recommendations for further preprocessing. 1. **Data standardization**: Since you've already applied standardization during the detection step (threshold=5.183), it's likely that your features are already in a similar scale. However, double-check if there are any extreme values or outliers. If so, consider re-standardizing your data to ensure all features have a similar range. 2. **Remove redundant variables**: With 10 numeric variables, some might be highly correlated. Use techniques like principal component analysis (PCA) or correlation analysis to identify strongly correlated variables. If two variables are highly correlated (>0.9), consider removing one of them to avoid redundant information and reduce dimensionality. 3. **Handle missing values**: Although you haven't mentioned any missing values, it's essential to check for any gaps in your data. If present, decide on a strategy to handle them: imputation (e.g., mean or median), interpolation, or removing rows with missing values. 4. **Explore and transform categorical variables (if any)**: Since there are no categorical variables mentioned, this step is not applicable. However, if you have categorical features in your dataset, consider encoding them using techniques like one-hot encoding, label encoding, or ordinal encoding. Before applying these preprocessing steps, I recommend re-running the PCA-based detection with the preprocessed data to evaluate its impact on the flagged window count and z-score values. Please let me know if you'd like me to elaborate on any of these points or if you have specific concerns about your dataset.

Type your question here... (Shift+Enter for new line)

Send

Conversation — ask in plain English and get a plain-English answer. Here the assistant responds to a preprocessing question with specific, dataset-aware advice. Use **Clear** to start fresh.

Analysis Context
What the assistant knows about your current analysis

Dataset: 1,500 rows × 10 columns

Numeric variables (10): AC1_PV, AC1_OP, FC1_PV, FC1_OP, PC1_PV, PC1_OP, PC2_PV, PC2_OP...

Pre-processing applied: Yes

Features extracted: 5 feature columns across 75 windows

Detection: PCA-based (Hotelling's T²) | 4/75 windows flagged (5.3%) | threshold=5.183

contamination=0.05 | standardized=Yes

Analysis Context — exactly what the assistant can see about your current work: dataset shape, variables, whether pre-processing ran, the features extracted, and the detection result. This is why its answers are specific to your run, not generic.

NOTE

The assistant is a convenience, not a requirement — the full pipeline (load → pre-process → features → detect → identify → report) works completely without it.